THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

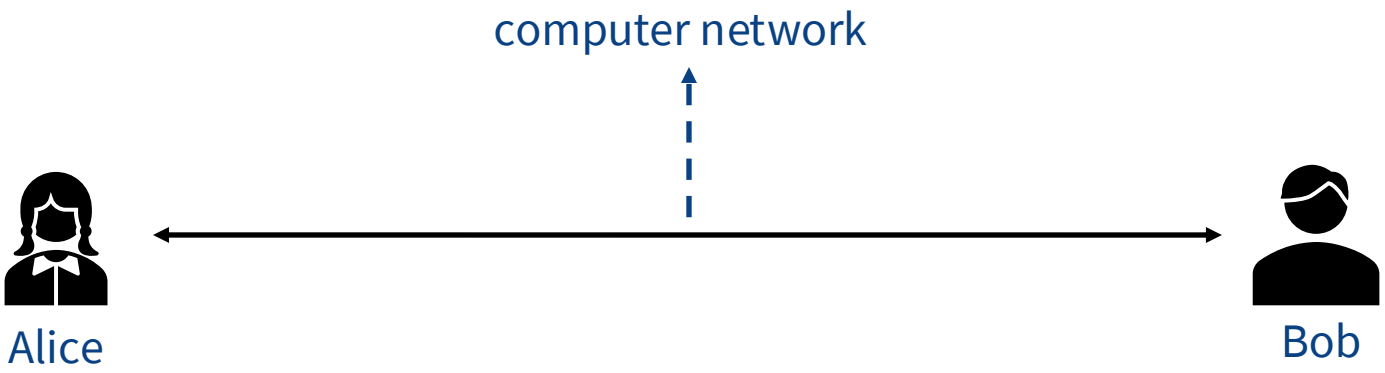
Secure Communication

Maria Rodriguez Read

Semester 1, 2025

© University of Melbourne

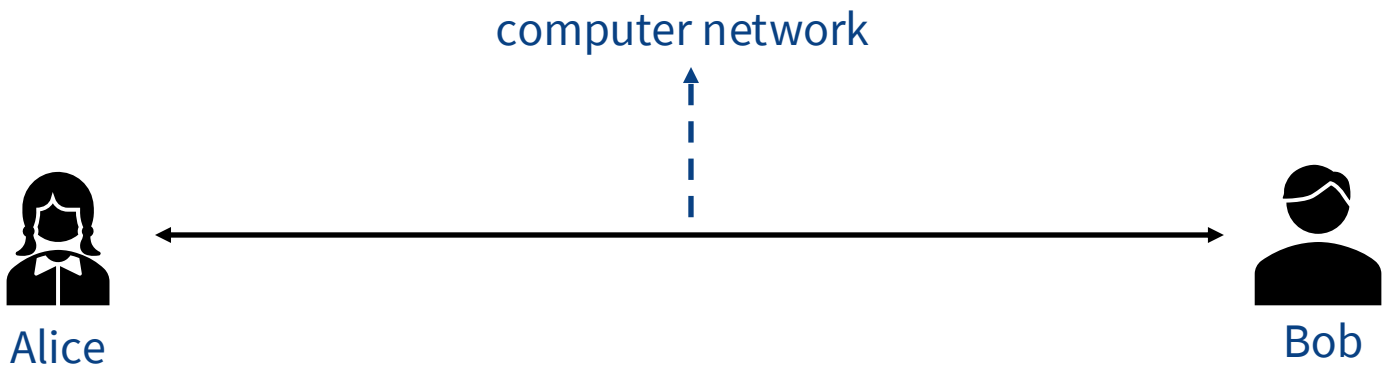
Secure Communication



- Alice and Bob want to communicate securely over a computer network
- Who are Alice and Bob?
  - Routers
  - A browser and a web server
  - The SSH client on your laptop and the SSH server on your VM
  - Git in your laptop and GitHub servers
  - Etc.

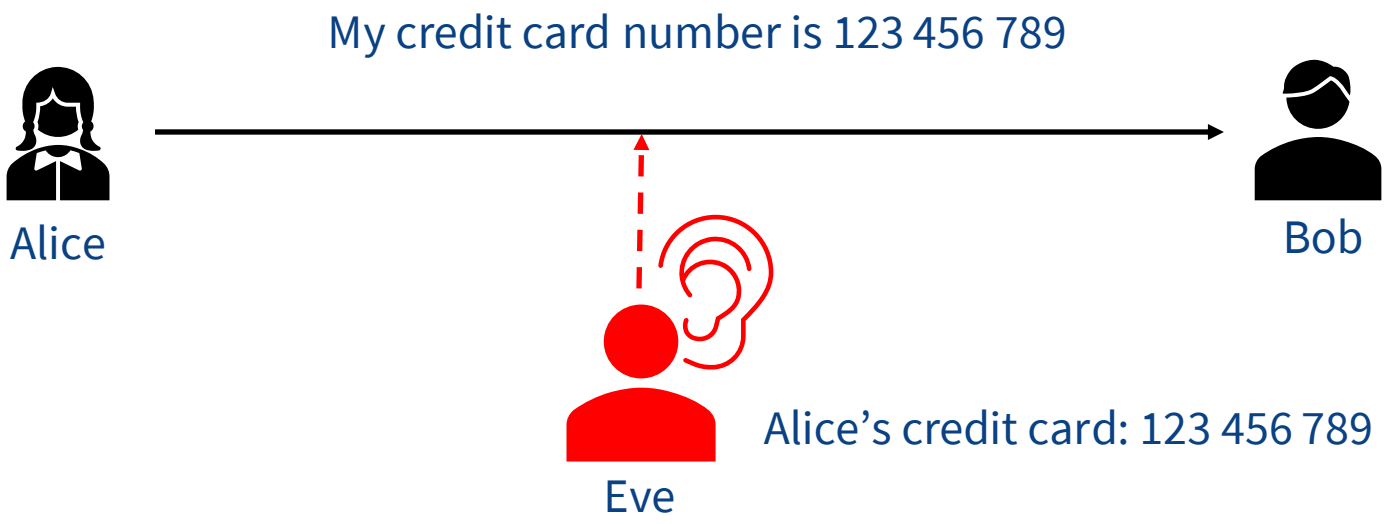
© University of Melbourne

Secure Communication



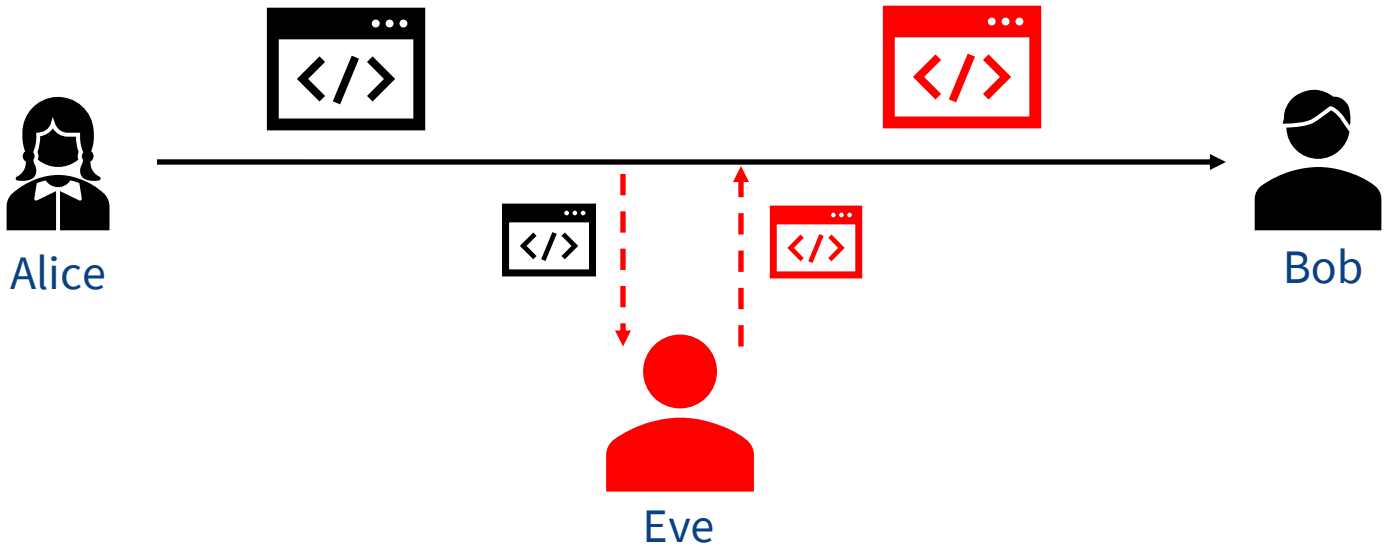
- Secure communication has the following properties:
  - **Confidentiality**
  - **Integrity**
  - **Authentication**

Secure Communication



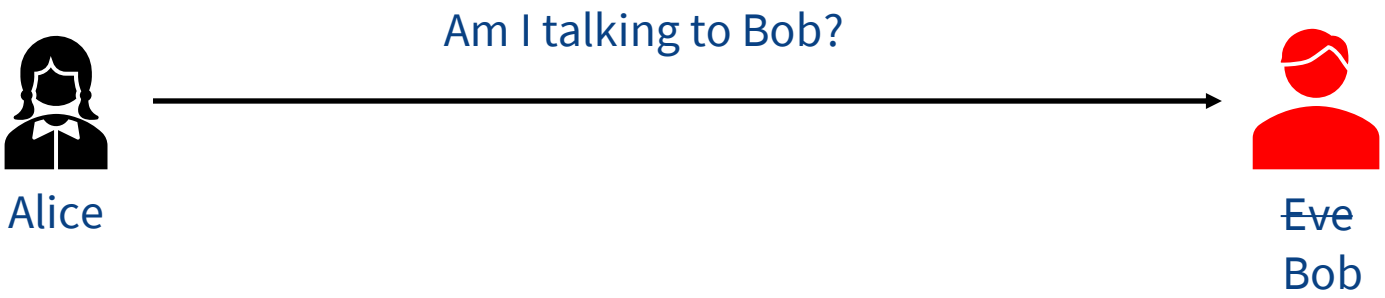
- **Confidentiality**
  - Only the sender and intended receiver should be able to understand the contents of the communication

Secure Communication



- **Integrity**
  - Detect if the content of the communication has been tampered with

Secure Communication



- **Authentication**
  - Establish the identities of one or both of the endpoints to confirm that they are who they claim to be

Cryptographic Principles

- Cryptographic techniques underpin secure communication:
  - Encryption
  - Cryptographic hashing
  - Message authentication codes
  - Digital signatures
- Modern cryptography is based on mathematics
  - Problems that are considered to be computationally hard, e.g.,
    - Factoring the product of 2 large primes (RSA)
    - Solving discrete logs (ElGamal)
- Cryptography is not absolute – there is no perfect security
  - Always susceptible to brute force attack
  - The challenge is to make the brute force attack take so long as to be infeasible to perform within the useful lifetime of the data

© University of Melbourne

Encryption

- Used to provide **confidentiality**
- Take a message, called the **plaintext**, and **encrypt** it into a **ciphertext** in such a way that only authorised parties know how to **decrypt** it back to plaintext
- Two functions
  - *c = encrypt(m,* K_e*):* encrypt message (plaintext) *m* using an encryption key K_e
  - *m = decrypt(c,* K_d*):* decrypt ciphertext *c* using a decryption key K_d
- The contents of the message are hidden from everyone except those holding the decryption key
- Kerckhoffs' Principle
  - Secrecy should depend only on the decryption key remaining secret
  - Secrecy should not depend upon the algorithm remaining secret

© University of Melbourne

Encryption

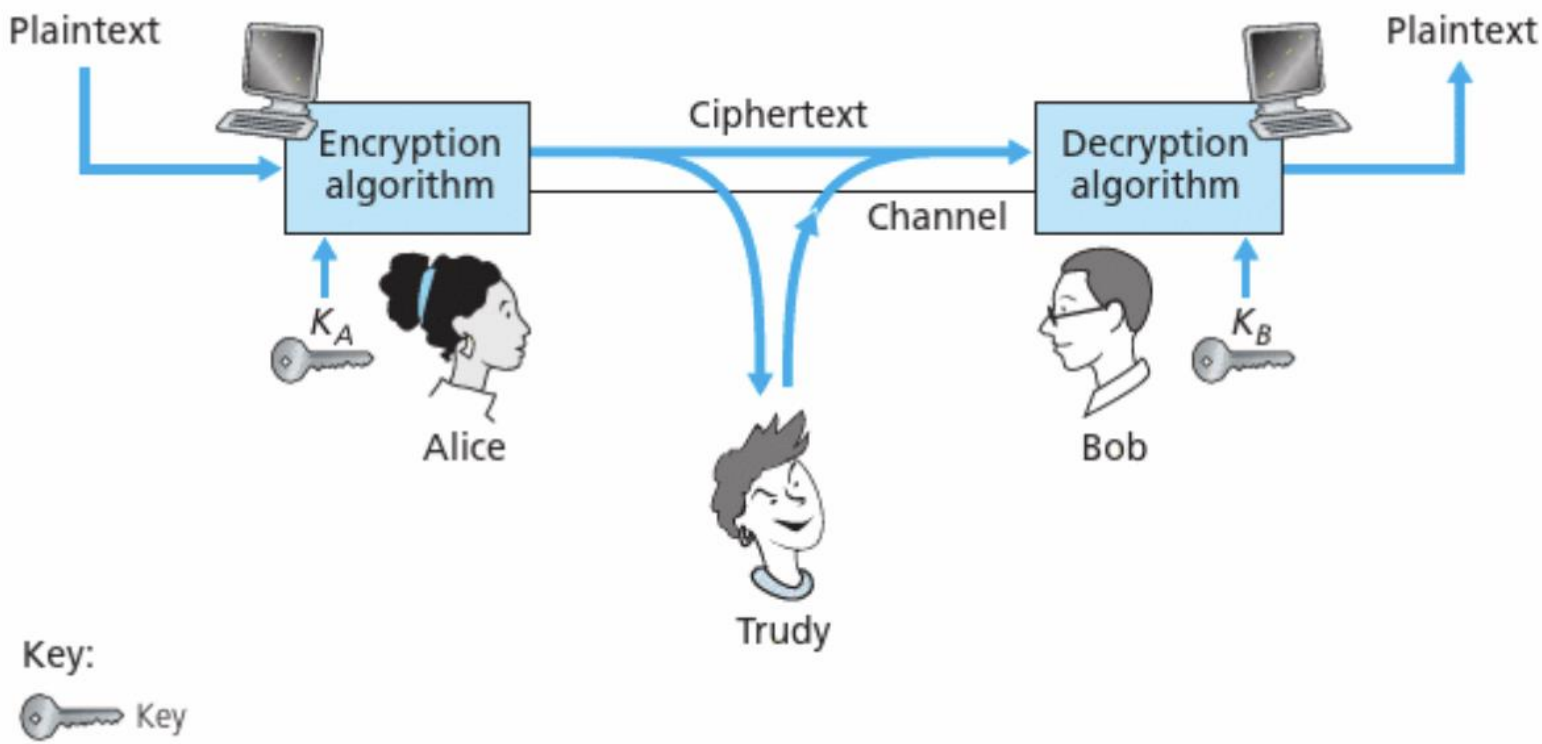


Figure 8.2 ♦ Cryptographic components

Symmetric and Asymmetric Encryption

- Symmetric encryption
  - Uses the **same key** for encryption and decryption
  - The key must remain secret
- Asymmetric encryption (also known as public key encryption)
  - Uses **two different keys**
  - A public key to encrypt and a private key to decrypt
  - The private key must remain secret

© University of Melbourne

Symmetric Encryption

- $c = encrypt(m, K_s)$
- $m = decrypt(c, K_s)$
- Relies on the secret key K_s being securely exchanged between the communicating parties
- A modern example is AES (Advanced Encryption Standard)

© University of Melbourne

Symmetric Encryption Protocol

- Alice wants to send a message to Bob
  1. Alice and Bob securely exchange K_s
  2. Alice computes ciphertext *c* by running *encrypt(m,* K_s*)*
  3. Alice sends *c* to Bob
  4. Bob recovers *m* by computing *decrypt(c,* K_s*)*
- Provides **confidentiality** provided K_s is kept secret
- Problem: how to securely exchange K_s?

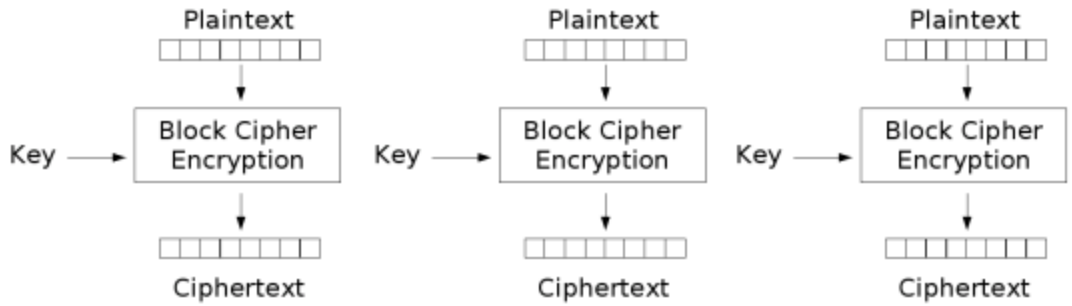
Advanced Encryption Standard - AES

- AES is a block cipher
  - Breaks data into fixed-sized blocks and encrypts/decrypts each block
- AES, and block ciphers in general, have different modes of operation
  - The mode of operation determines how each block is treated and/or linked
  - Examples of the mode of operation include:
    - ECB – Electronic Codebook
    - CBC – Cipher Block Chaining

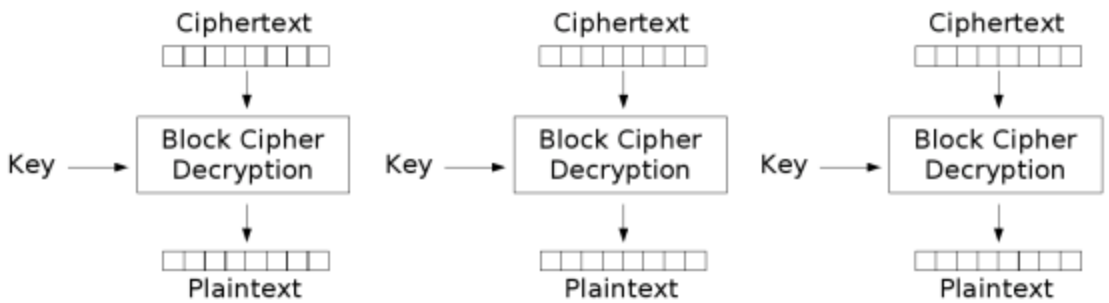
© University of Melbourne

ECB – Electronic Codebook

- Each block is encrypted and decrypted independently using K_s



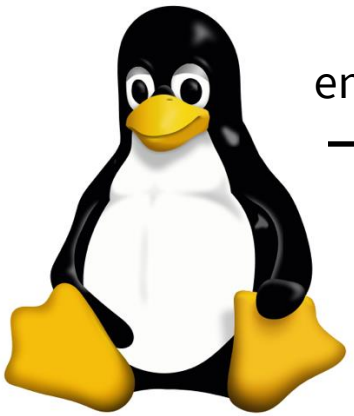
Electronic Codebook (ECB) mode encryption



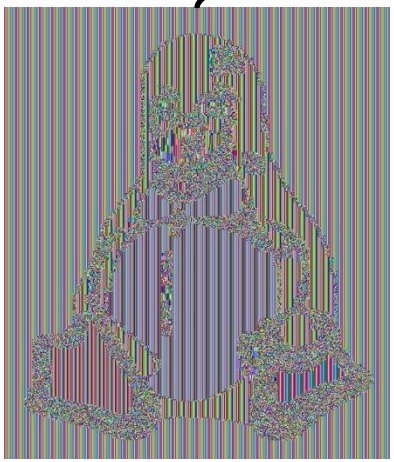
Electronic Codebook (ECB) mode decryption

ECB – Electronic Codebook

- Simple, both encryption and decryption are parallelisable
- Generally, it should never be used:
  - The same text always encrypts to the same cipher (deterministic encryption)
  - Leaks information about the plaintext
    - 2 identical plaintext blocks result in 2 identical ciphertext blocks
    - 2 identical messages result in identical ciphertexts



encrypted under ECB →



Repeated patterns will be evident in the output, albeit at a lower fidelity

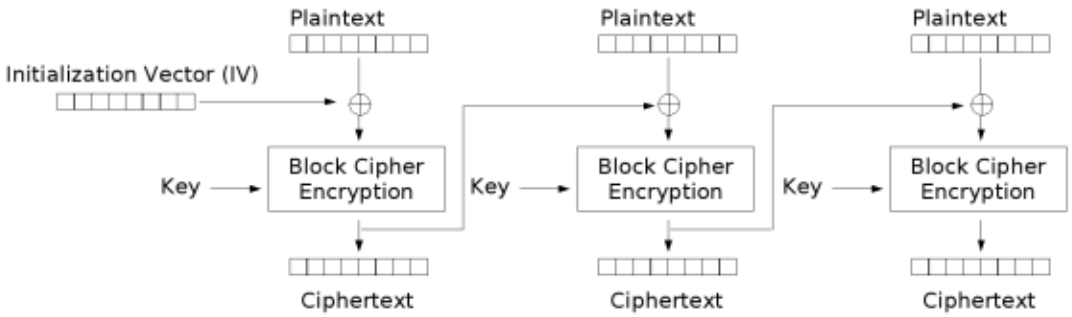
CBC – Cipher Block Chaining

- Based on probabilistic encryption
  - The use of randomness during encryption so that encrypting the same plaintext results in different cyphertexts
- When using CBC, we cannot conclude that because two ciphertexts are different, then the plaintexts were different
- In CBC, randomness is introduced through an Initialization Vector (IV)
  - Must be random and not reused
  - Does not need to be secret

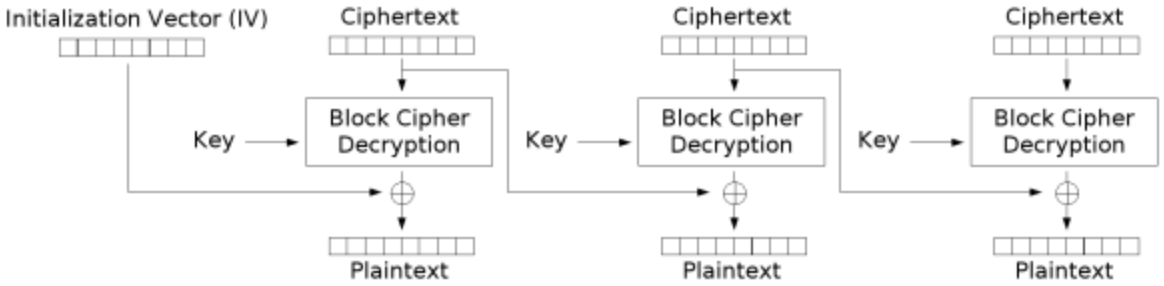
© University of Melbourne

CBC – Cipher Block Chaining

- Encryption
  - First block: XOR plaintext and IV, encrypt, get ciphertext
  - Other blocks: XOR previous ciphertext with plaintext, encrypt, get ciphertext
- Decryption
  - First block: Decrypt ciphertext and XOR with IV, get plaintext
  - Other blocks: Decrypt ciphertext and XOR with previous ciphertext, get plain text



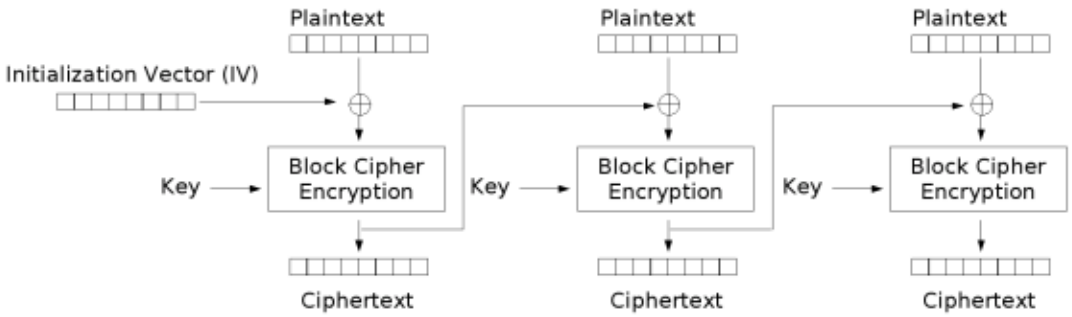
Cipher Block Chaining (CBC) mode encryption



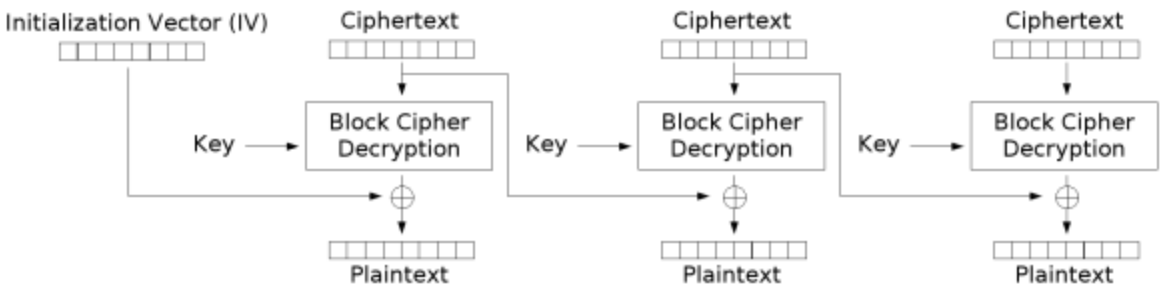
Cipher Block Chaining (CBC) mode decryption

CBC – Cipher Block Chaining

- Encryption must be done sequentially
  - Encrypting block n requires encrypting block n-1 first
- Decryption can be done in parallel
  - No need to decrypt block n-1 before decrypting block n



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

Asymmetric Encryption

- Also known as public key encryption
- Every principal (i.e., Alice, Bob) has a pair of keys: a public one and a private one
- **Encryption is done with the public key**, which can be made available to anyone
  - $c = encrypt(m, K_{public})$
- **Decryption is done with the private key**, which must remain secret (i.e., only accessible to the owner of the key pair)
  - $m = decrypt(c, K_{private})$
- Examples of algorithms include RSA, ElGamal

© University of Melbourne

Asymmetric Encryption Protocol

- For Alice to send a message to Bob:
  1. Bob generates a key pair $(K^B_{public}, K^B_{private})$
  2. Bob posts his public key online
  3. Alice accesses Bob's public key and computes ciphertext c by running $encrypt(m, K^B_{public})$
  4. Alice sends c to Bob
  5. Bob recovers m by computing $decrypt(c, K^B_{private})$
- There is no need to securely exchange any keys
- Provides confidentiality provided:
  - $K^B_{private}$ is kept secret
  - K^B_{public} is Bob's public key
- Problems:
  - Asymmetric encryption is not suitable for encrypting large messages, it is computationally expensive

Hybrid Encryption

- So far
  - Symmetric encryption is efficient for encrypting large messages but requires securely exchanging a secret key
  - Asymmetric encryption does not require securely exchanging a secret key but is not suitable for encrypting large messages
- How can we combine both approaches to derive a protocol that provides confidentiality efficiently for large messages?
  - Use asymmetric encryption to securely exchange a shared secret key
  - Use symmetric encryption to encrypt communication messages using the shared secret key

© University of Melbourne

Key Exchange Protocol using Asymmetric Encryption

- Alice and Bob want to exchange messages using symmetric key encryption
  1. Alice generates a key pair $(K^A_{public}, K^A_{private})$
  2. Alice posts her public key online
  3. Bob obtains K^A_{public}
  4. Bob generates a secret key K_{secret}
  5. Bob computes the ciphertext of the secret key $c = encrypt(K_{secret}, K^A_{public})$
  6. Bob sends c to Alice
  7. Alice recovers K_{secret} by computing $\mathrm{de}crypt(c, K^A_{private})$
  8. Alice and Bob share a secret key and can exchange messages using symmetric encryption

© University of Melbourne

Towards Secure Communication

- So far, we have learnt how to achieve confidentiality efficiently through a hybrid of symmetric and asymmetric encryption
- But how about integrity? how do we verify that a message wasn't tampered with?

© University of Melbourne

Message Integrity

- Verify that a message was not tampered with
- Why do we need to protect the integrity of messages?
  - An attacker can read, modify, and delete messages
- Two approaches:
  - Message Authentication Code (MAC)
  - Digital Signatures
- Before we look at these approaches, we need to understand cryptographic hashing

© University of Melbourne

Cryptographic Hashing

- A (plain old) hash function takes an arbitrary length input *m* and produces a fixed length hash, or digest, *H(m)*
- A cryptographic hash function has additional properties:
  - Collision resistance
    - Collision: $H(m)=H(m')$ for some inputs $m \neq m'$
    - Collision resistance: hard to find $m \neq m'$ such that $H(m)=H(m')$
  - One-way
    - Given $H(m)$, hard to find m
- Example functions include SHA-2, SHA-3

© University of Melbourne

Cryptographic Hashing

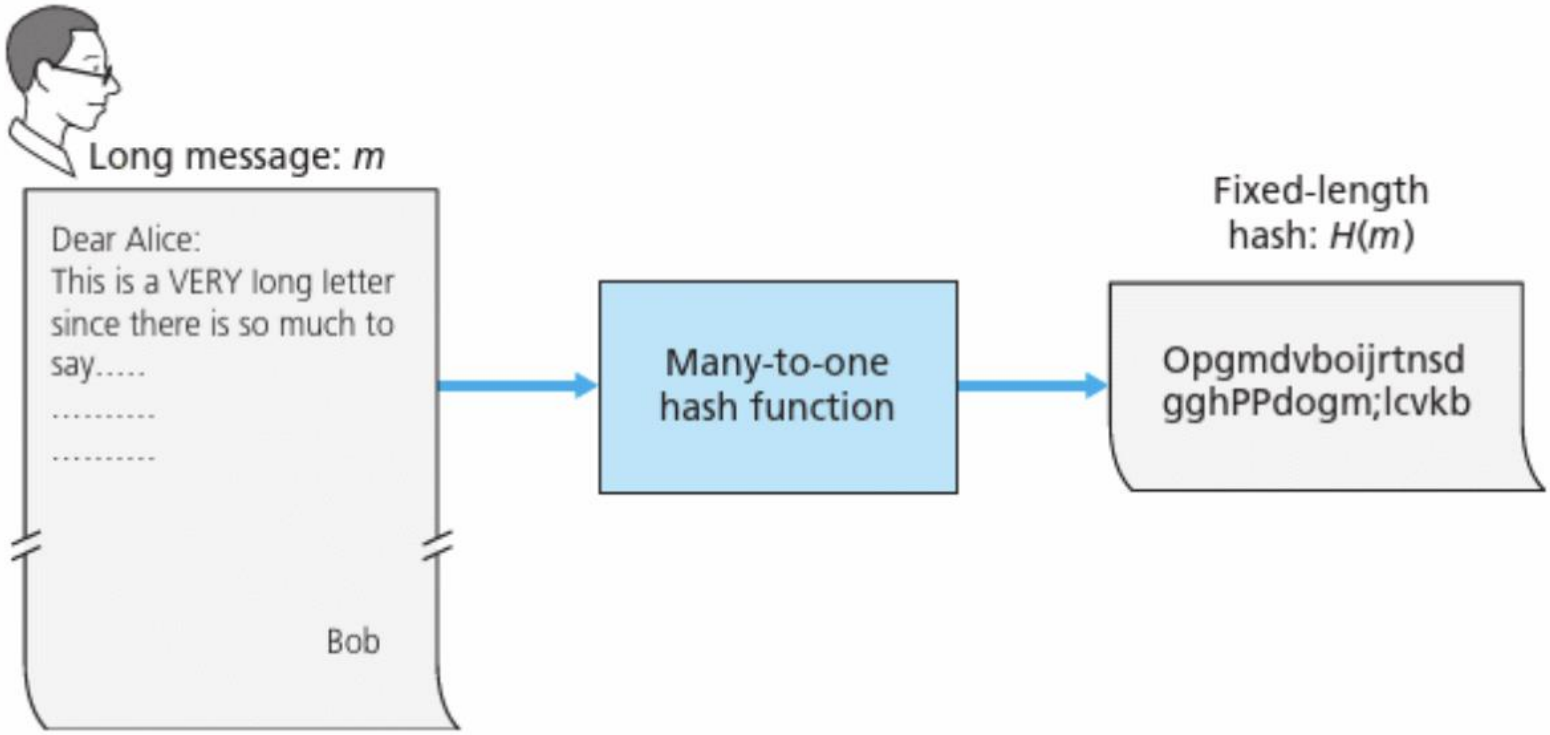


Figure 8.7 ♦ Hash functions

Message Authentication Code (MAC)

- Used to detect if a message has been tampered with
- Requires a shared secret key (symmetric cryptography)
- A MAC function
  - Generates a tag *t* given a message and a secret key
    - $t = mac(m, K_s\ )$
  - Is typically a cryptographic hash function that takes as input the secret key and the message concatenated somehow, e.g.,
    - $H(\ m \| s)$
    - $H((s \oplus opad) \| H((s \oplus ipad) \| m))$ [HMAC]

MAC Message Exchange Protocol

- Alice and Bob have a shared secret key K_s and want to verify the integrity of the messages they exchange
  1. Alice generates a message *m* she wants to share with Bob
  2. Alice generates a tag $t = mac(m, K_s)$
  3. Alice sends *m* and *t* to Bob
  4. Bob calculates $t' = mac(m, K_s)$
  5. Bob verifies $t' == t$. If the tags match, *m* has not been tampered with. If they don't match, *m* has been tampered with
- An adversary cannot forge the tag of a message without knowledge of key
- The message is sent in plaintext: no protection of confidentiality
- The goal is to detect modification

MAC

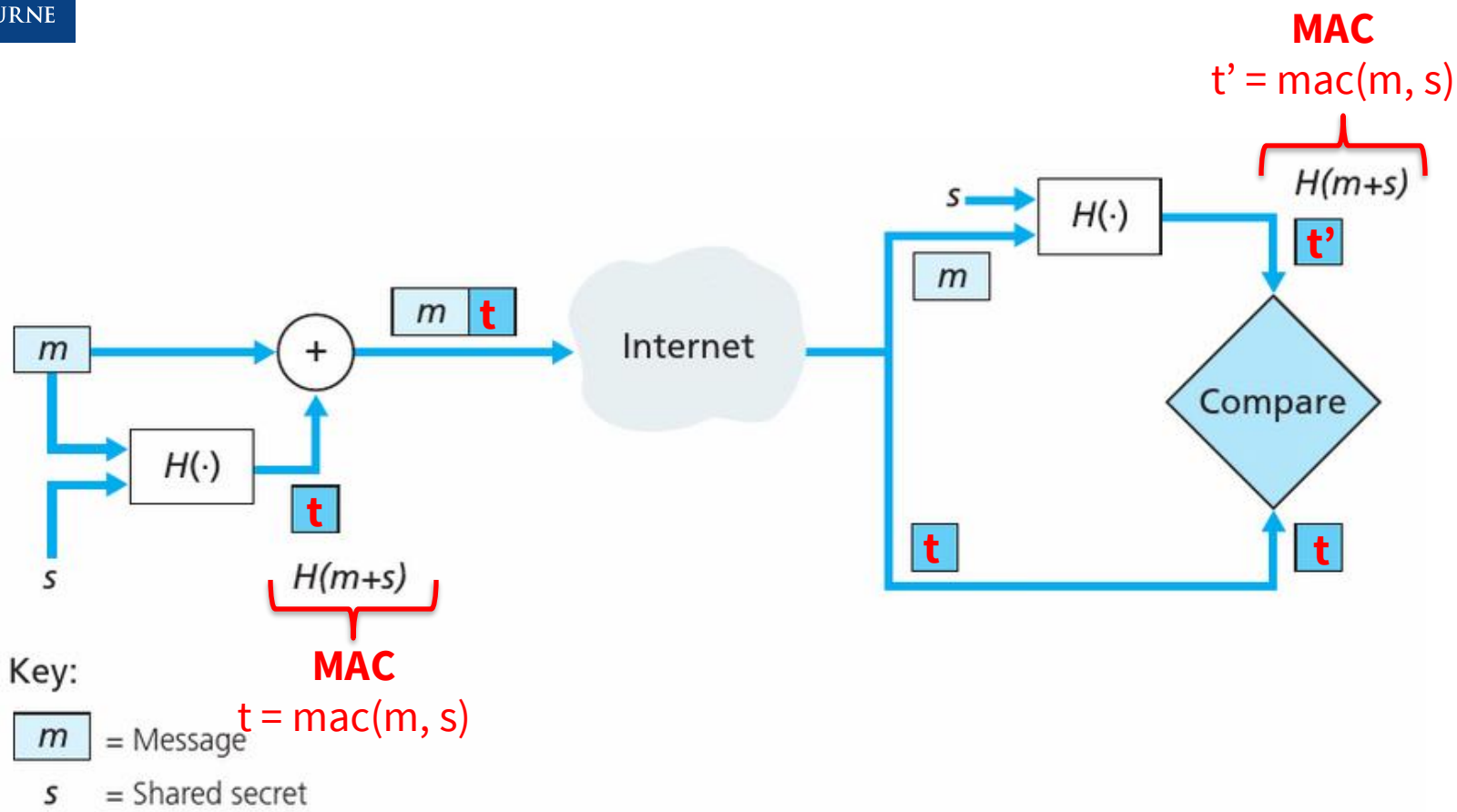


Figure 8.9 ♦ Message authentication code (MAC)

Digital Signatures

- Asymmetric cryptography:
  - A private signing key
  - A public verification key
- Signature algorithm
  - Produce a signature for a message using the signing key
  - $s = sign(m, K_{sign})$
- Verification algorithm
  - Verify a signature given the verification key, the signature and the original message
  - $verify(m, s, K_{ver})$
- Provide integrity and non-repudiation (signer cannot deny signing the document)
- If Bob signed a message and verification is successful:
  - The message must have been signed using Bob's private signing key
  - Only Bob has access to his private signing key, so it must have been Bob who signed the message (non-repudiation)
  - The original message has not been modified (integrity)

© University of Melbourne

Digital Signature Protocol

- Bob wants to verify a message was sent by Alice and that it wasn't tampered with. Bob has access to Alice's public verification key
  1. Alice generates a signature for message *m*, *s* = $sign(m, K_{sign}^{A})$
  2. Alice sends *m* and *s* to Bob
  3. Bob accepts the message only if $verify(m, s, K_{ver}^{A})$ succeeds
- Message is sent in plaintext, no confidentiality
- Goal is to detect modification
- What if *m* is too long for asymmetric algorithms?

Digital Signatures with Hashing

- For large messages
  - Generate the hash digest *H(m)* of the message
  - Sign and verify *H(m)*
- Protocol:
  1. Alice generates *s* = $sign(H(m), K_{sign}^{A})$
  2. Alice sends *m* and *s* to Bob
  3. Bob generates $H(m)$ and accepts the message only if $verify(H(m), s, K_{ver}^{A})$ succeeds

© University of Melbourne

Towards Secure Communication

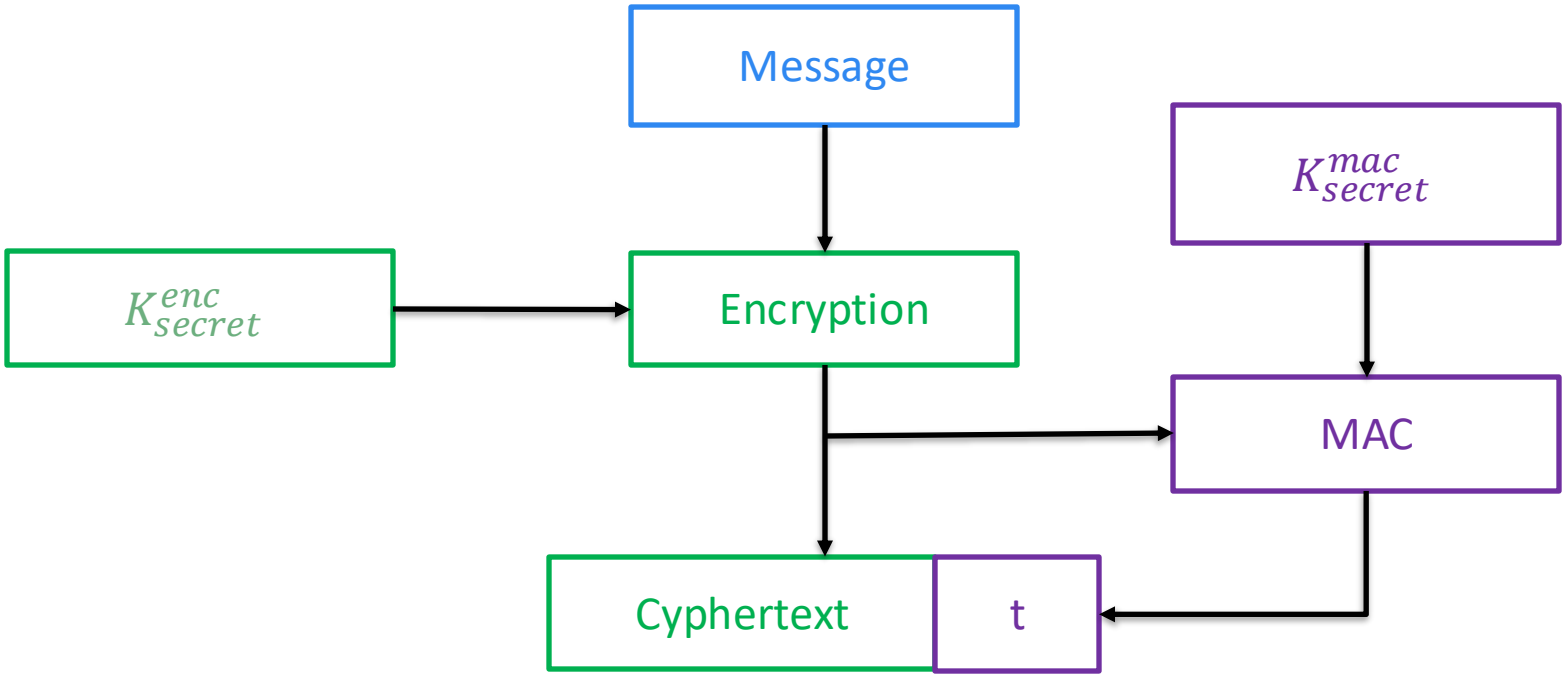
- So far, we have learnt cryptographic principles that support either confidentiality or integrity
- But what if we want both?
  - Authenticated encryption

© University of Melbourne

Authenticated Encryption

- Provides confidentiality and integrity of messages
- Approach: encrypt then MAC
- Protocol:
  1. Alice generates $c = encrypt(m, K_{secret}^{enc})$
  2. Alice generates a tag for c, $t = \text{mac}(c, K_{secret}^{mac})$
  3. Alice sends c and t to Bob
  4. Bob generates the tag for c, $t' = \text{mac}(c, K_{secret}^{mac})$
  5. If $t == t'$ then $m = \text{de}crypt(c, K_{secret}^{enc})$, else abort
- Examples: AES-GCM, AES-OCB, AES-CCM

Encrypt then MAC



Properties provided: confidentiality and integrity

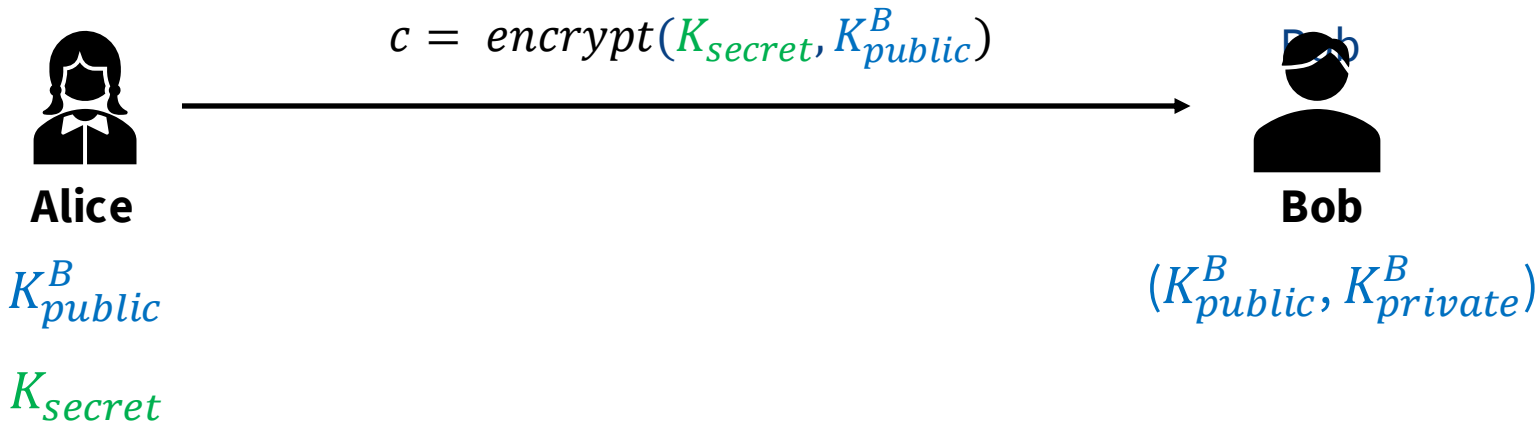
Towards Secure Communication

- Through encrypt then MAC, we can achieve integrity and confidentiality
- Encrypt then MAC is a symmetric cryptosystem, it relies on shared keys being securely exchanged
  - Shared secret key to encrypt/decrypt
  - Shared secret key to generate MAC tags
- How can we securely exchange keys?
  - Recall our asymmetric encryption key exchange protocol
  - But…there is one more issue that needs to be addressed for the protocol to be secure: authentication

© University of Melbourne

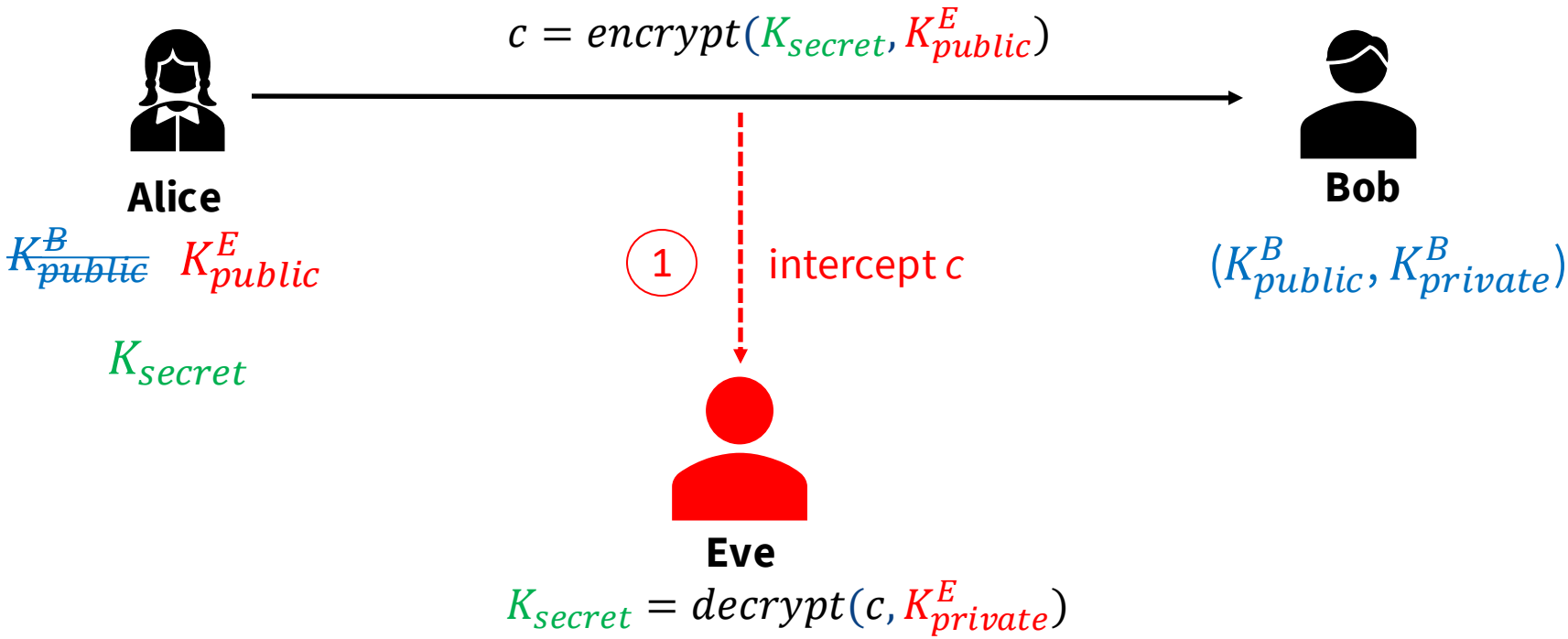
Towards Secure Communication

Asymmetric encryption key exchange protocol



How does Alice know K^B_{public} is Bob's public key?

Towards Secure Communication



Towards Secure Communication

$$c' = encrypt(K_{secret}, K^B_{public})$$

$$\cancel{c = encrypt(K_{secret}, K^E_{public})}$$

Alice

$\cancel{K^B_{public}}$ K^E_{public}

K_{secret}

2 replace c with c'

Bob

$(K^B_{public}, K^B_{private})$

$K_{secret} = decrypt(c, K^B_{private})$

Eve

$K_{secret} = decrypt(c, K^E_{private})$

We need a way of verifying that Bob is the owner of the public key

Certificates

- Digital certificates securely associate identities with cryptographic public keys
- A digital certificate is a signature binding together:
  - Identity of principal
  - Public key of that principal (e.g., encryption or verification key)
- Given a *certificate issuer* with a key pair $(K_{public}^{issuer}, K_{private}^{issuer})$, a principal's identity (*Bob*), and their public key (K_{public}^{Bob}):
  - $binding = (Bob, K_{public}^{Bob})$
  - *signature* = $sign(binding, K_{private}^{issuer})$
  - $\boldsymbol{certificate(Bob, issuer) = (binding, signature)}$
- Issuer is certifying that K_{public}^{Bob} belongs to Bob

Authentication with certificate

- Sample protocol to verify Bob's identity and public key
  1. Bob creates a message *m* = "I am Bob"
  2. Bob signs the message: $s_m = sign(m, K_{private}^{Bob})$
  3. Bob sends $m, s_m, certificate(Bob, issuer)$ to Alice
  4. Alice verifies the certificate by
     a. Verifying the certificate's signature: $verify(binding, signature, K_{public}^{issuer})$
     b. Verifying that the identity in *binding* corresponds to Bob's identity
  5. If verification is successful, Alice retrieves K_{public}^{Bob} from certificate
  6. Accept if $verify(m, s_m, K_{public}^{Bob})$ succeeds
- Problem: how can we trust the authenticity of K_{pubic}^{issuer}?

Certificate Authorities

- Certificate authorities are entities that are explicitly trusted
- Sign certificates for others
- Their public keys are contained in *root certificates* that are self-signed
- Root certificates are shipped, pre-loaded, with your Operating System/Browser

© University of Melbourne

Transport Layer Security (TLS)

- **TLS is NOT EXAMINABLE**
- Protocol for secure communication over the Internet
- Supported by all popular web browsers , web servers, internet commerce sites
- HTTPS: implementation of HTTP over TLS

© University of Melbourne

TLS Basics

- Handshake protocol:
  - Uses public-key cryptography to establish several shared secret keys between the client and the server
  - An initial negotiation between client and server that establishes the parameters of their subsequent interactions within TLS
- Record protocol:
  - Uses the secret keys established in the handshake protocol to protect confidentiality, integrity, and authenticity of data exchange between the client and the server

© University of Melbourne

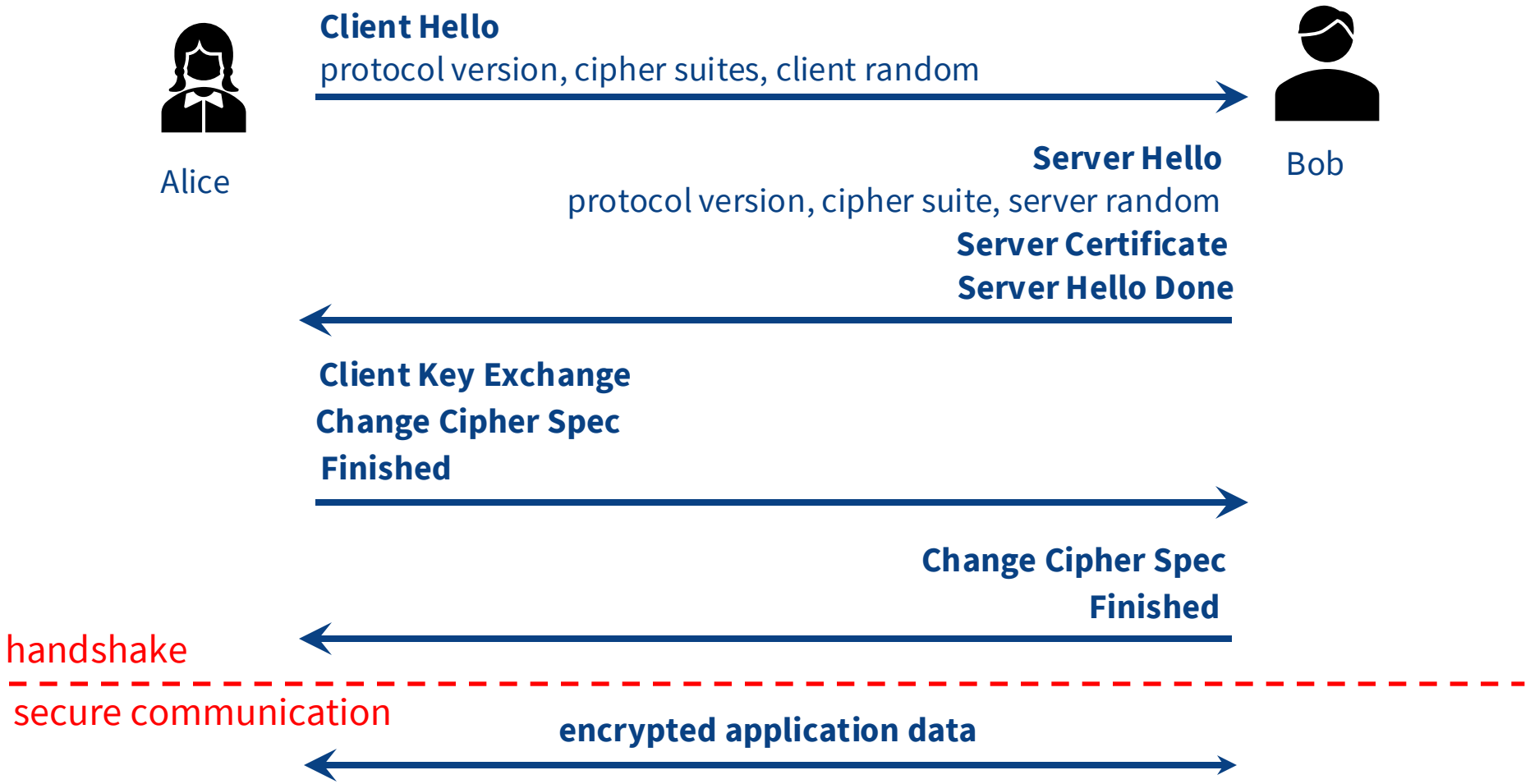
Handshake Protocol

- Runs between a client and a server
  - For example, client = browser, server = web server

1. **Negotiate** version of the protocol and the set of cryptographic algorithms to be used:
   - Interoperability between different implementations
2. **Authenticate** server and client (optional)
   - Use digital certificates to learn each other's public keys and verify each other's identity
   - Often, only the server is authenticated
3. **Establish a shared secret**

© University of Melbourne

Sample Handshake (TLS 1.2)



© University of Melbourne 2022

Sample Handshake (TLS 1.2)

- **Client Hello:** The client asks the server for a secure connection, lists its supported cipher suites, and sends a client random (random bytes)
- **Server Hello:** The server selects one of the cipher suites and sends a server random
- **Server Certificate:** contains the public key certificate of the server (and any other supporting certificates in the certificate chain)
- **Server Hello Done:** This message means that the server is done sending messages to support the key exchange, and the client can proceed with its phase of the key exchange. Upon receipt of this message, the client verifies the validity of the server's certificate
- **Client Key Exchange:** the client generates a premaster secret and sends the encrypted premaster secret (with the server's public key) to the server. The client and the server use the premaster secret and the client and server randoms to generate the secret keys needed to secure the communication (e.g., symmetric encryption and MAC key)
- **Change Cipher Spec: L**ets the other party know that the session key has been generated and encrypted communication communication will be used from now on
- **Finished:** The first message to be protected with the negotiated algorithms, keys, and secrets. Once a side has sent its Finished message and received and validated the Finished message from its peer, it may begin to send and receive application data over the connection

© University of Melbourne

Sample Handshake (TLS 1.3)

Alice

Bob

ClientHello
Protocol version
Cipher suites
Key share (e.g., g^x)

ServerHello
Protocol version
Cipher suite
Key share (e.g., g^y)

Certificate
CertificateVerify (digital signature)
Finished (MAC)

Finished (MAC)

TLS 1.3 does not support RSA key exchange.
Instead, key exchange is done using the Diffie-Helman key exchange algorithm (provides forward secrecy)

© University of Melbourne 2022

Sample Handshake (TLS 1.3)

- Client sends ClientHello to server asking for secure connection, listing its supported cipher suites. The client assumes the server will support a given key exchange protocol (e.g., DH) and sends the public component of the key (e.g., g^x)
- Server responds with ServerHello and selects one of the cipher suites presented that it supports, sends its corresponding public component of the key (e.g., g^y)
- Server sends a Certificate message containing its public key certificate and any other supporting certificates in the certificate chain.
- Server sends a Certificate CertificateVerify message containing a signature over the entire handshake using the private key corresponding to the public key in the Certificate message.
- Server sends a Finished message which consists of a MAC over the entire handshake.
- Client validates: certificate, signature, and MAC
- Client sends a Finished message consisting of the MAC of the entire handshake
- Server validates MAC
- Handshake concludes and both parties can use symmetric cryptography to communicate securely